

Elktech Exercice n°2

Hugging Face • MLflow • Optuna • Core ML

Context

The objective of this exercise is to explore the complete lifecycle of a modern AI product, from model selection to deployment within an iOS application.

You will develop an application capable of analyzing an image, detecting the objects it contains, and automatically displaying the number of detected objects per category. For example, an image containing four people, two cars, and one bicycle should produce a result showing these counts while also displaying the corresponding bounding boxes.

Beyond model performance, this exercise is designed to familiarize you with modern AI engineering practices, including experiment reproducibility, configuration management, automated hyperparameter optimization, and deployment on Apple devices.

Assignment

You must select an object detection model from the Hugging Face ecosystem (DETR, YOLOS, RT-DETR, or an equivalent model) and build a complete pipeline for training, evaluation, optimization, and deployment.

The entire project must be fully configurable. No hyperparameters should be hardcoded in the source code. All model, dataset, training, and inference parameters must be defined in a YAML configuration file that serves as the single source of truth for the entire pipeline.

The goal is for any experiment to be fully reproducible using only the source code, the YAML configuration file, and the corresponding MLflow run.

Expected Architecture

The Hugging Face model must be treated as a fully interchangeable component. By changing only the model name in the configuration file, it should be possible to switch from one model architecture to another without modifying the rest of the pipeline.

Optuna must drive the hyperparameter optimization process by automatically generating and evaluating different YAML configurations. At a minimum, the search space should include learning rate, batch size, image size, weight decay, and inference thresholds.

All experiments must be tracked in MLflow, including parameters, metrics, artifacts, and the YAML configuration used for each run. The objective is to enable systematic comparison of experiments and identification of the best-performing configuration.

Apple Deployment

Once the best model has been identified, it must be exported to Core ML and integrated into an iOS application built with SwiftUI.

The application should allow users to capture a photo or select an image from their photo library, run the model locally on the device, and display the detected objects, their locations within the image, and the total object count for each category.

Particular attention should be paid to on-device performance, including inference latency, model size, and prediction quality.

Deliverables

Each team must submit:

- A Git repository containing the complete training, optimization, and deployment pipeline.
- A functional iOS application.
- A short technical report describing the selected model, the MLflow experiments, the Optuna optimization process, the results obtained, the Core ML deployment workflow, and the main limitations encountered.
- The MLflow interface
- A short demonstration video showcasing the complete application workflow.

Benchmarking Requirement

A key objective of this exercise is not only to train a model, but also to build a systematic evaluation framework for comparing multiple object detection architectures.

Each team is therefore required to evaluate and compare several Hugging Face object detection models. The comparison must be conducted through a reproducible benchmarking pipeline that enables objective analysis of model performance across multiple dimensions, including detection quality, inference speed, model size, and suitability for mobile deployment.

The final deliverable must include a benchmarking dashboard or report summarizing the results of all evaluated models and providing a clear justification for the final model selection.

The selected model should not simply be the model with the highest accuracy. Teams are expected to analyze and discuss the trade-offs between prediction quality, latency, memory footprint, and deployment constraints within the Apple ecosystem.

The purpose of this exercise is to learn how to:

- treat a Hugging Face model as a configurable component;
- design reproducible experimentation pipelines using MLflow;
- automate hyperparameter optimization with Optuna;
- objectively compare multiple model architectures;
- make data-driven model selection decisions;
- deploy optimized AI systems within the Apple ecosystem using Core ML.

By the end of the exercise, the team should be capable of building a complete AI engineering workflow, from experimentation and benchmarking to production deployment on iOS devices.

Evaluation Criteria

Evaluation will focus as much on engineering quality as on model performance. Particular attention will be given to project architecture, experiment reproducibility, the effective use of MLflow and Optuna, the quality of the Core ML deployment, and the team's ability to justify and analyze their technical decisions.